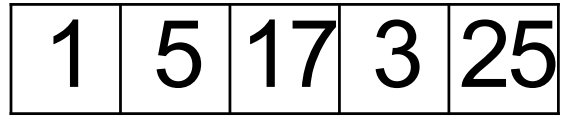
Array



Definition

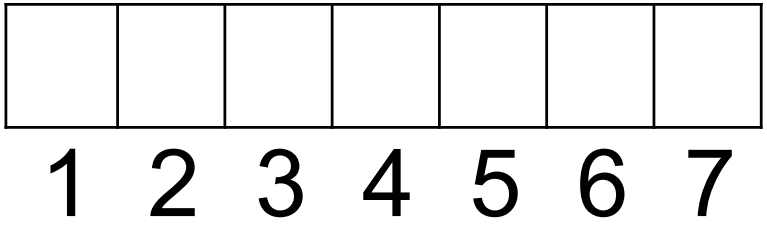
Array:

Contiguous area of memory

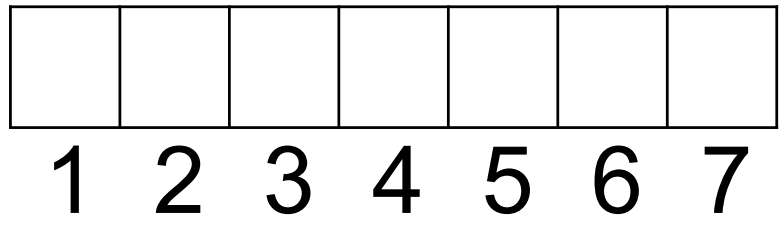
Definition

Array:

Contiguous area of memory consisting of equal-size elements indexed by contiguous integers.

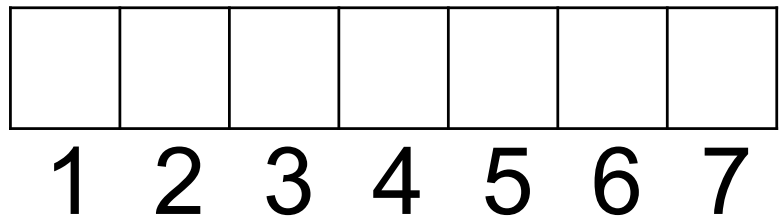


What's Special About Arrays?



What's Special About Arrays?

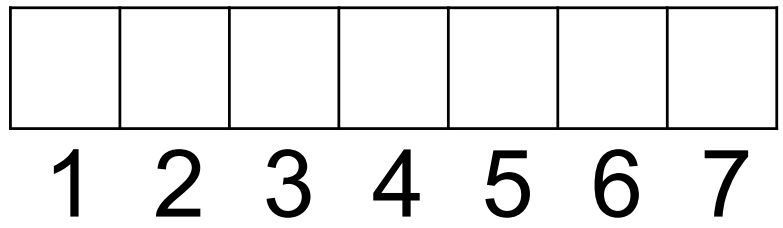
Constant-time access



What's Special About Arrays?

Constant-time access

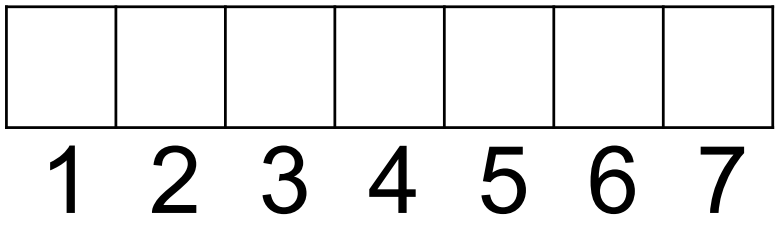
array_addr



What's Special About Arrays?

Constant-time access

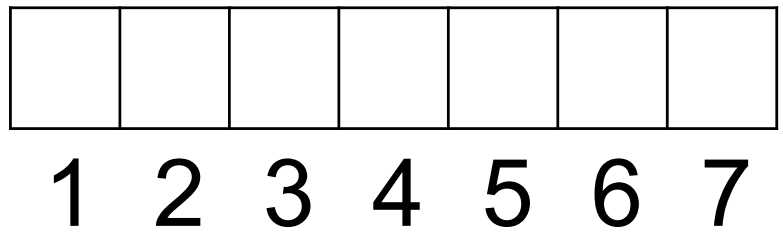
array_addr + elem_size × ( )



What's Special About Arrays?

Constant-time access

array_addr + elem_size × (*i* − first_index)



Question

Given an array whose:
•address is 1000,
•element size is 8
•first index is 0
What is the address of the element at index 6?

40

48

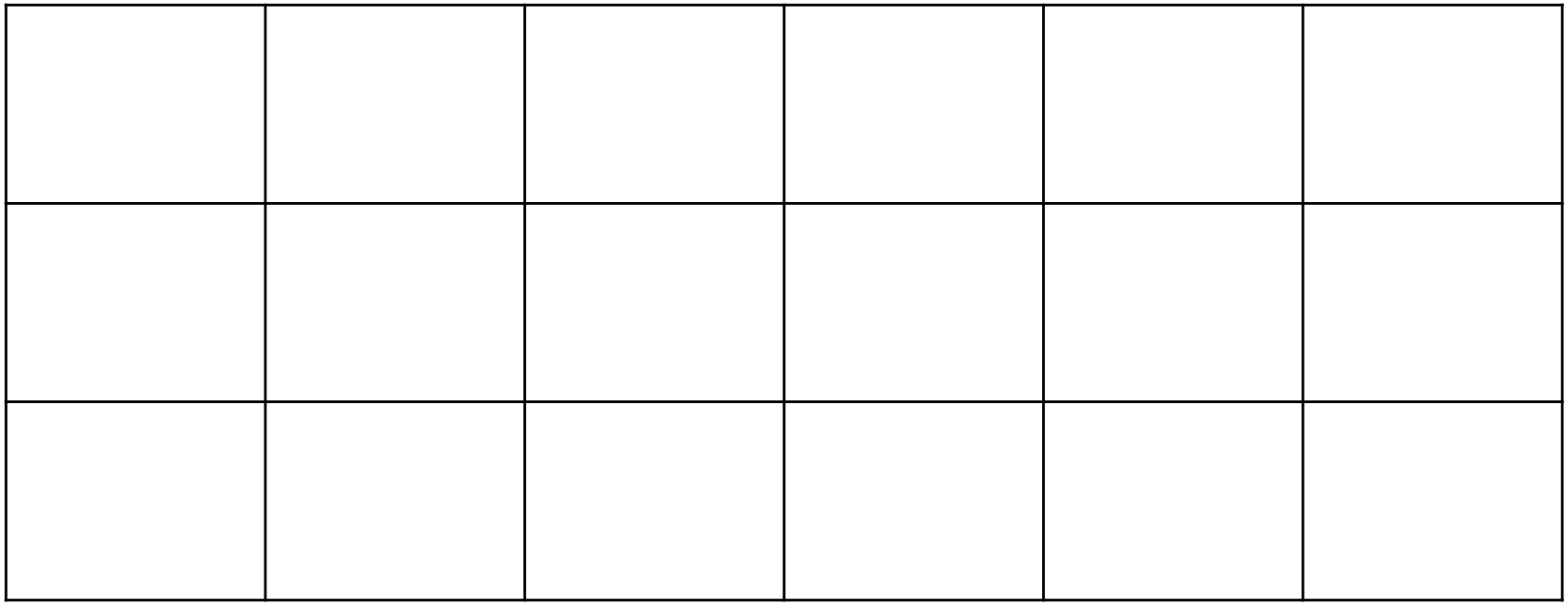
1048

1040

1006

1005

Multi-Dimensional Arrays



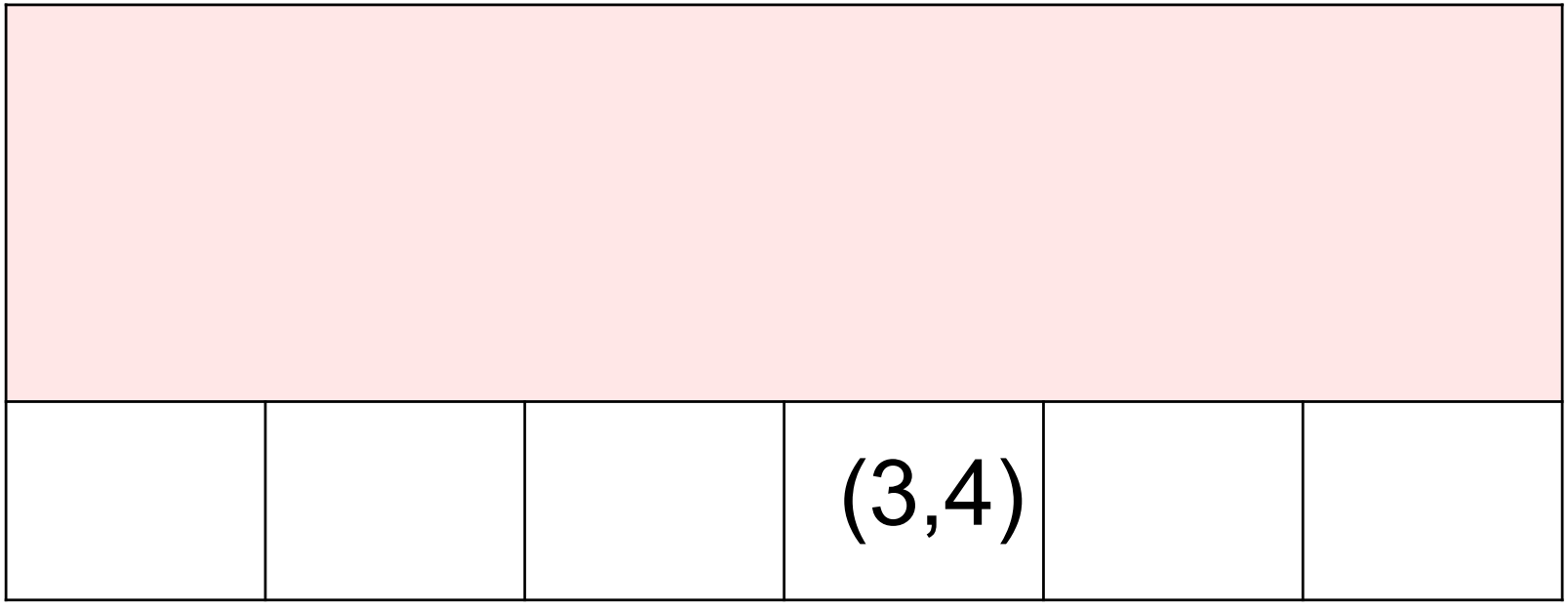
Multi-Dimensional Arrays

(1, 1)					

Multi-Dimensional Arrays

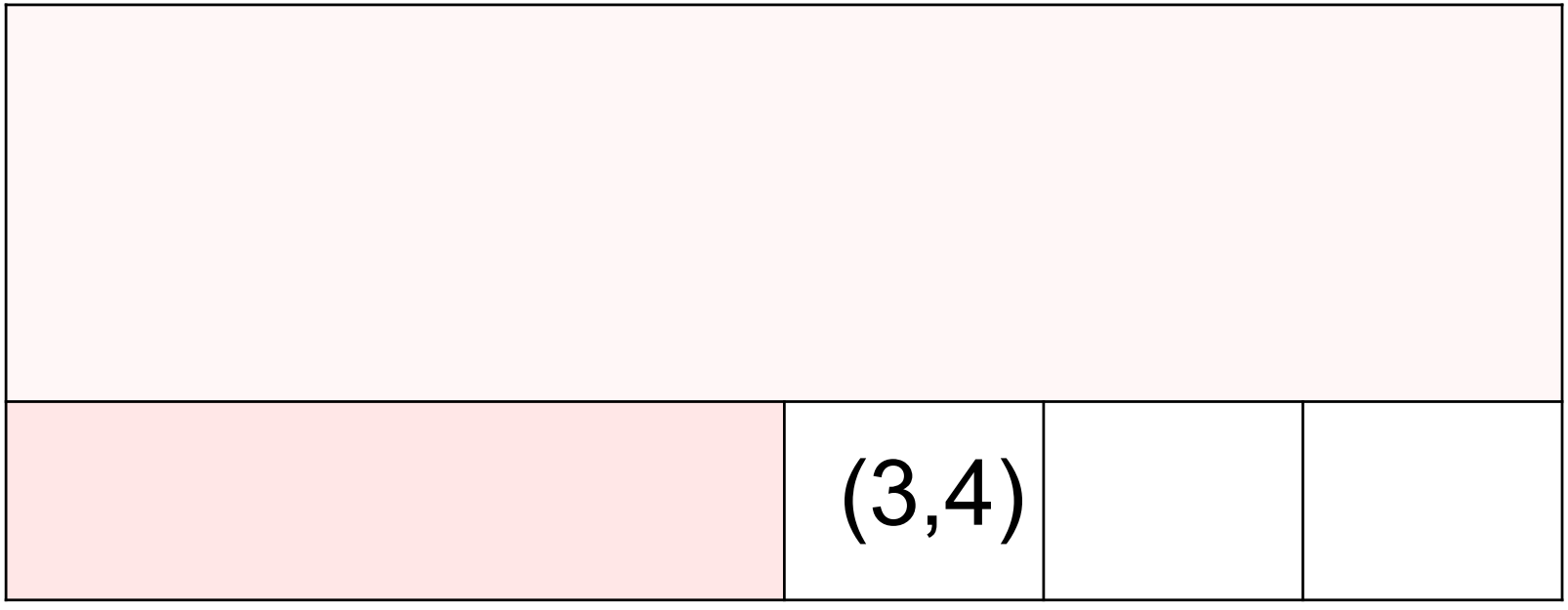
			(3,4)		

Multi-Dimensional Arrays



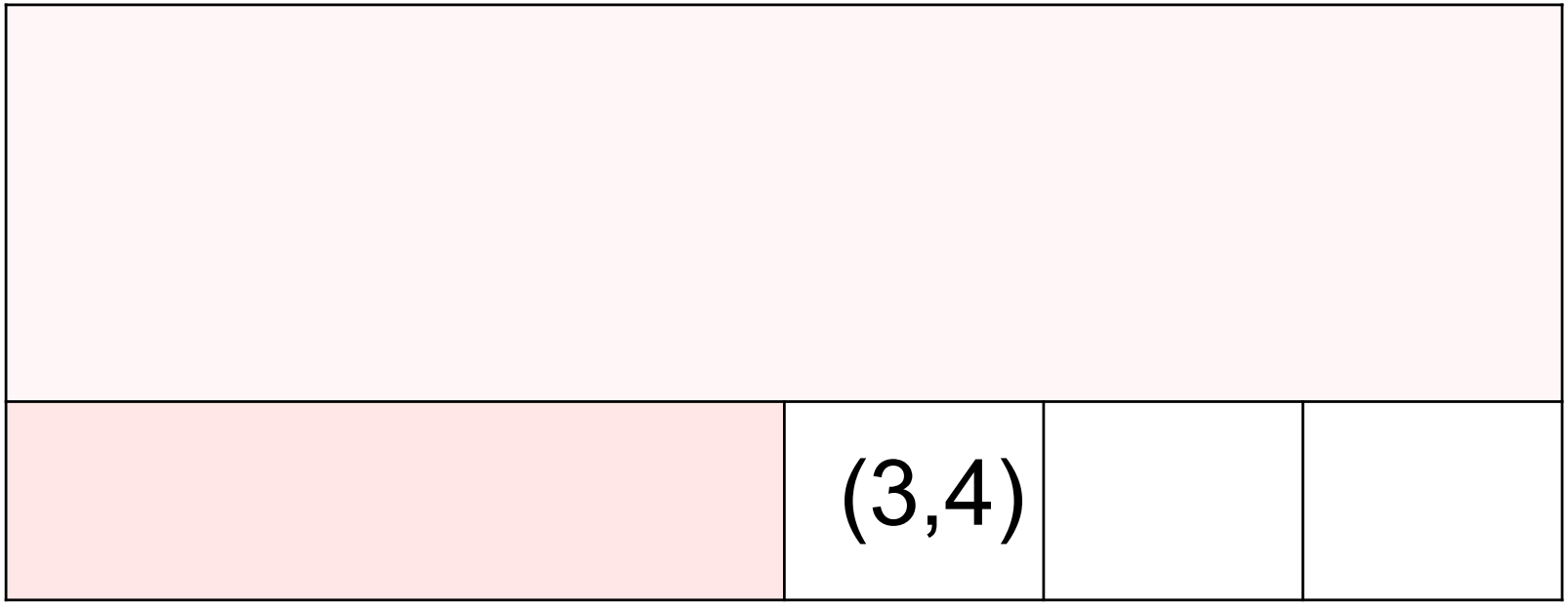
$(3 - 1) \times 6$

Multi-Dimensional Arrays



$$(3 - 1) \times 6 + (4 - 1)$$

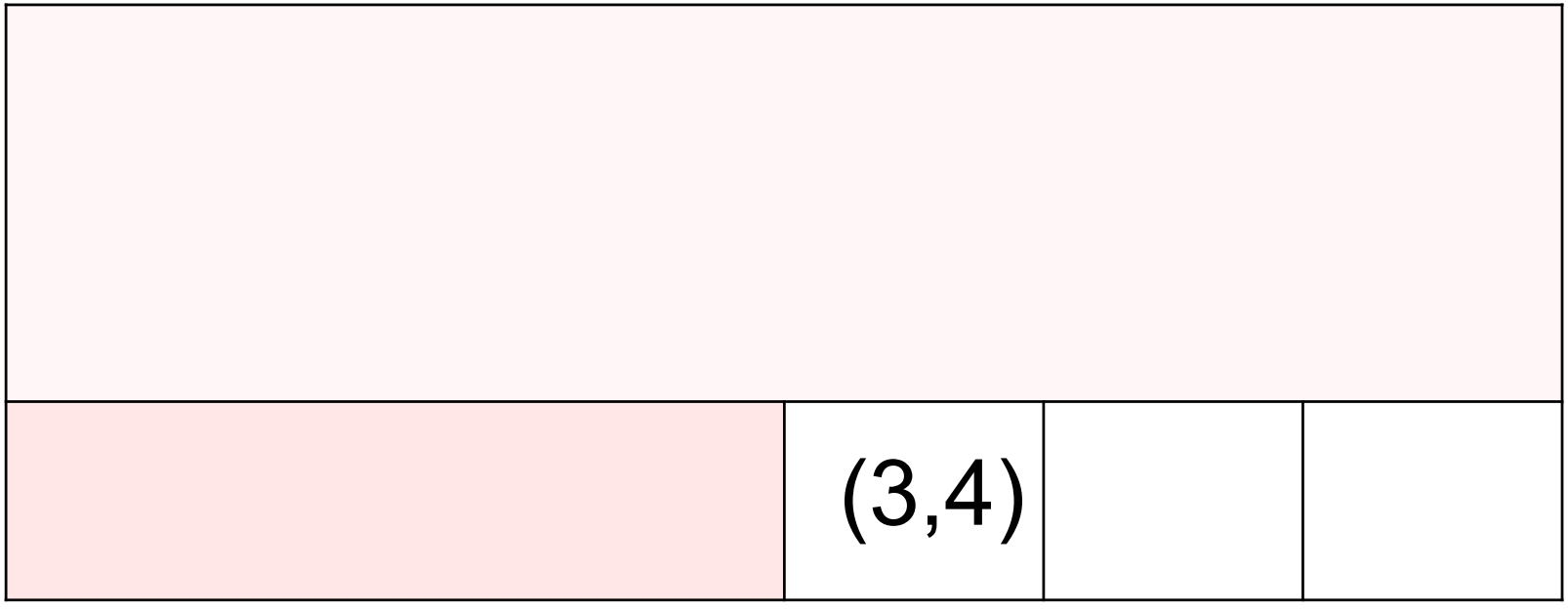
Multi-Dimensional Arrays



address_element = Array_address + elem_size × ((3-1)×no of column + (4-1))

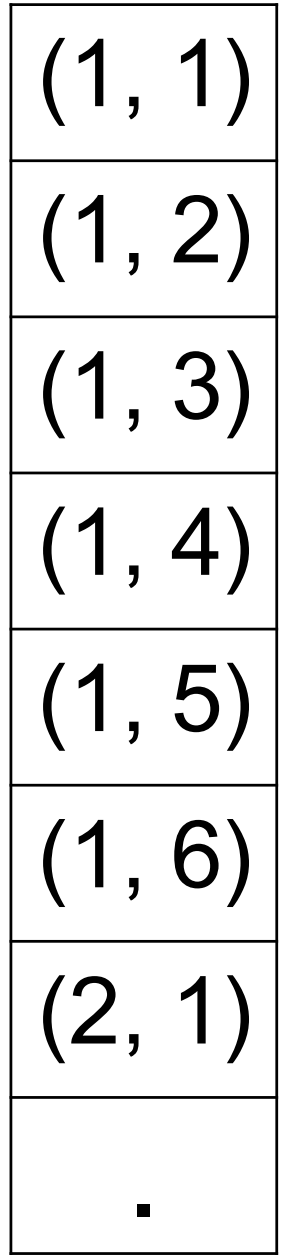
$$elem\_size \times ((3-1) \times 6 + (4-1))$$

Multi-Dimensional Arrays

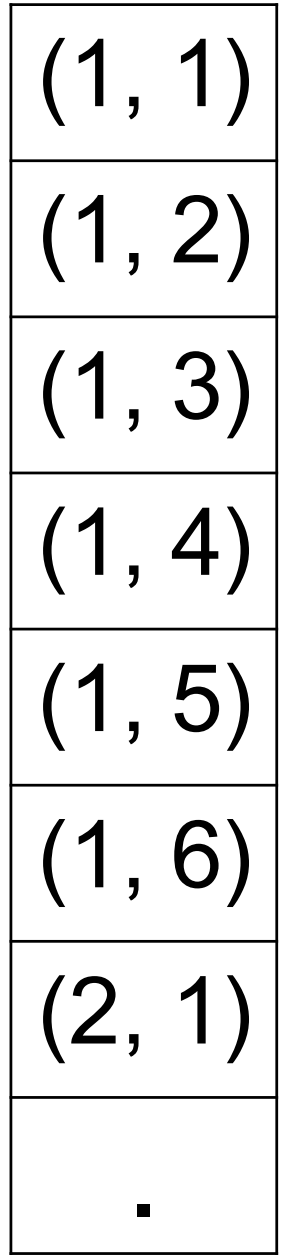


array_addr +

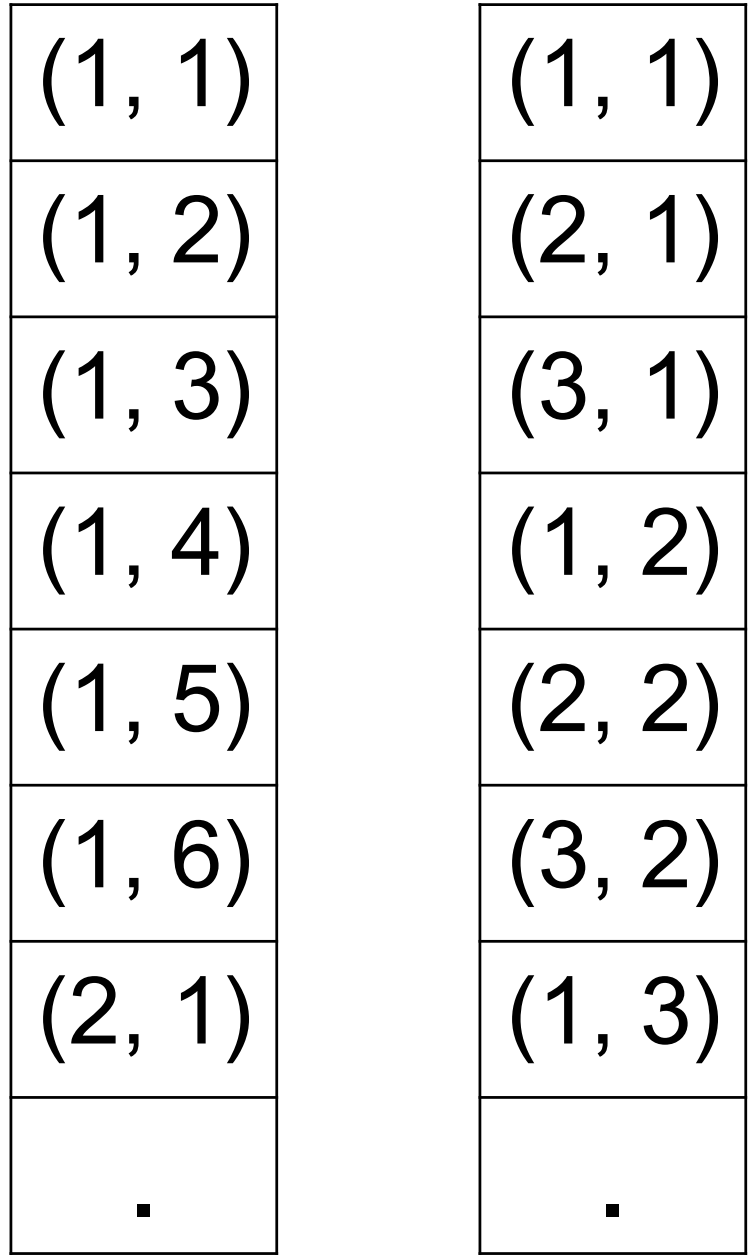
elem_size × ((3 − 1) × 6 + (4 − 1))

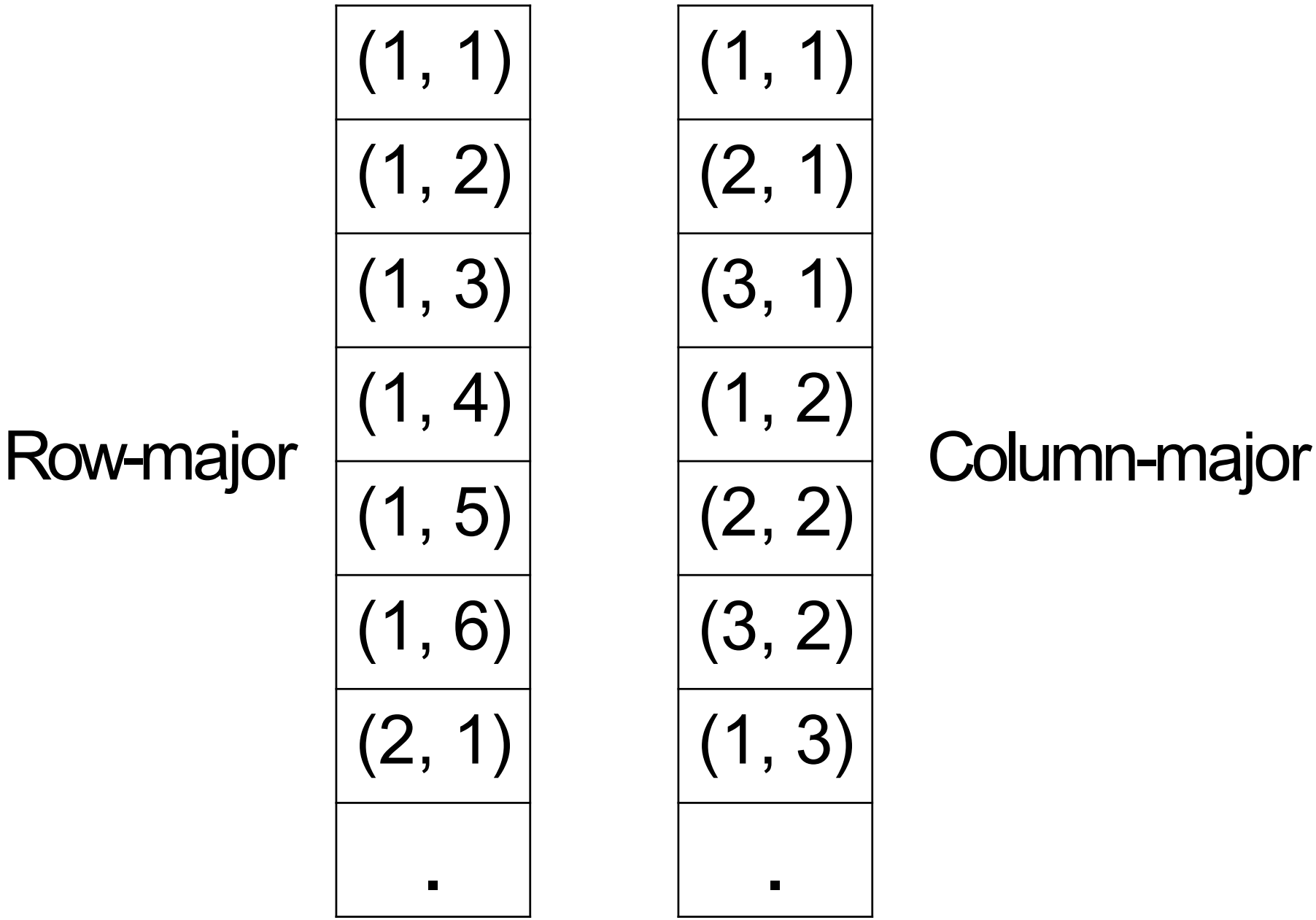

(1, 1)
(1, 2)
(1, 3)
(1, 4)
(1, 5)
(1, 6)
(2, 1)
.

Row-major



Row-major



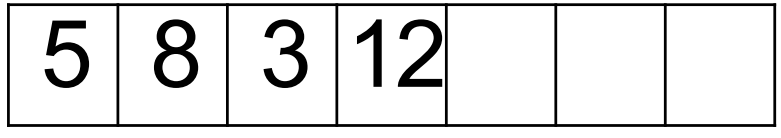

Row-major
(1, 1)
(1, 2)
(1, 3)
(1, 4)
(1, 5)
(1, 6)
(2, 1)
.
(1, 1)
(2, 1)
(3, 1)
(1, 2)
(2, 2)
(3, 2)
(1, 3)
.
Column-major

Times for Common Operations

	Add	Remove
Beginning		
End		
Middle		

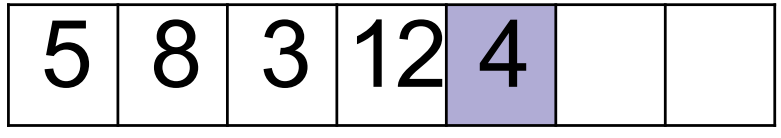
Times for Common Operations

	Add	Remove
Beginning		
End		
Middle		



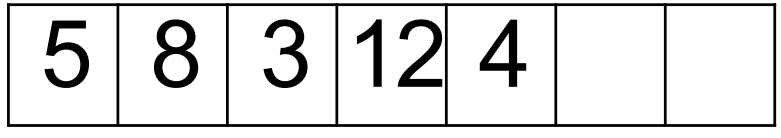
Times for Common Operations

	Add	Remove
Beginning		
End	*O*(1)	
Middle		



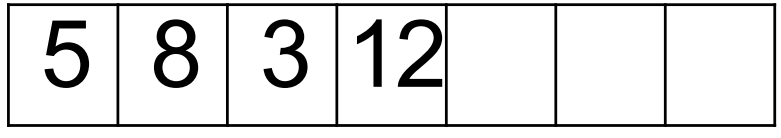
Times for Common Operations

	Add	Remove
Beginning		
End	*O*(1)	
Middle		



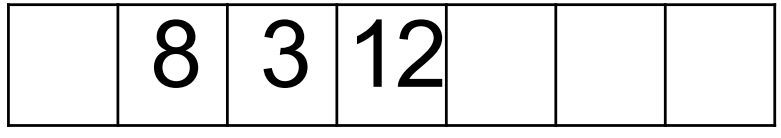
Times for Common Operations

	Add	Remove
Beginning		
End	$O(1)$	$O(1)$
Middle		



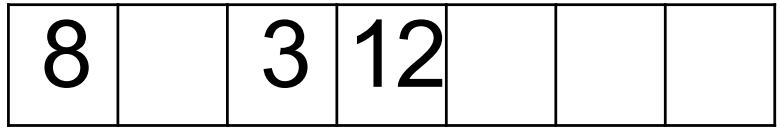
Times for Common Operations

	Add	Remove
Beginning		$O(n)$
End	$O(1)$	$O(1)$
Middle		



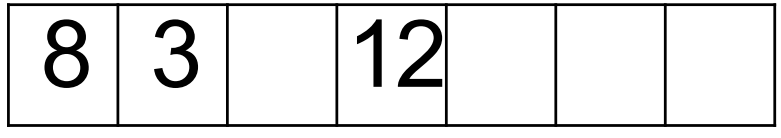
Times for Common Operations

	Add	Remove
Beginning		$O(n)$
End	$O(1)$	$O(1)$
Middle		



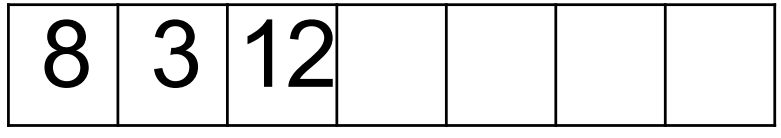
Times for Common Operations

	Add	Remove
Beginning		$O(n)$
End	$O(1)$	$O(1)$
Middle		



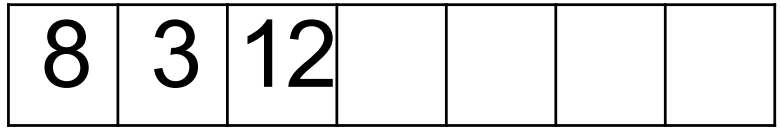
Times for Common Operations

	Add	Remove
Beginning		$O(n)$
End	$O(1)$	$O(1)$
Middle		



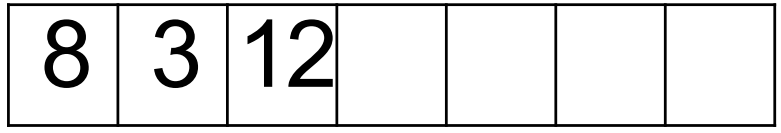
Times for Common Operations

	Add	Remove
Beginning	$O(n)$	$O(n)$
End	$O(1)$	$O(1)$
Middle		



Times for Common Operations

	Add	Remove
Beginning	$O(n)$	$O(n)$
End	$O(1)$	$O(1)$
Middle	$O(n)$	$O(n)$



HW

1.What is the time complexity of removing an element from the end of an array and why?

2.Consider the array [8, 3, 12]. What is the time complexity of adding the number 5 at the second position, and why?

3.If you want to remove the first element in the array [8, 3, 12], what would be the time complexity and the reason for this complexity?

4.What would be the time complexity of inserting an element at the third position of the array [8, 3, 12] if the array is not full?

HW

4. Given the pseudocode below, what is the time complexity of adding a new element at the beginning of an array, and why?

```
function addAtBeginning(array, new_element)
    for i from length(array) to 1 do
        array[i] = array[i-1]    // Shift all elements to the right
    end for
    array[0] = new_element       // Insert new element at index 0
end function
```

HW

5. Examine the pseudocode below. What is the time complexity of removing an element from the end of an array and why?

```
function removeAtEnd(array)
    last_element = array[length(array) - 1]  // Access the last element
    array[length(array) - 1] = null          // Remove the last element
    return last_element
end function
```

HW

6. In the provided pseudocode, how is the 5th element of the array accessed in constant time (O(1)) using the formula array_addr + elem_size × (index - first_index)?

```
function accessElement(array_addr, elem_size, index, first_index)
    offset = elem_size * (index - first_index)
    element_address = array_addr + offset
    return element_at(element_address)
end function

array_addr = 1000      // Base address of the array
elem_size = 4          // Each element is 4 bytes
index = 5              // Access the 5th element
first_index = 0        // Arrays are 0-indexed

element = accessElement(array_addr, elem_size, index, first_index)
```

Next Class Linked List